TradeWatch: Concept Overview

In one sentence: feed an AI model real market history bar-by-bar (never letting it see the future), ask it whether price will go up, down, or stay flat, require it to be confidently right several times in a row before treating that as a real signal, then measure — in real dollars — whether trading on that signal would actually have made money.

This is a **shadow-testing harness**, not a trading bot. It never places a real order. It exists to answer one question honestly, before any real capital is at risk: *can this AI's confidence be trusted, and would acting on it have paid off?*

The problem this solves

An AI model (Jev, from TypeSafe.ai) can be asked to predict whether a stock or index will move up or down over the next few hours. That prediction is only useful if two separate things are both true:

1. **The model is actually right often enough, and knows how confident to be** (calibration) — not just occasionally right, but honestly right in proportion to how sure it claims to be.
2. **A disciplined trading rule built on top of that signal — entries, exits, position sizing — actually turns a profit after realistic filtering**, not just "the model called the direction correctly."

Most AI-trading ideas fail because neither of these gets checked before real money goes on the line. This system checks both, on real historical (and live) market data, before anything is trusted.

Why it's built this way

1. **No peeking at the future.** Every prediction is computed from only the market data available up to that moment — recent volatility and trend, nothing later. This is the single most important rule: a backtest that leaks future information into its own predictions will always look great and always fail in reality.
2. **The AI is asked a structured question, not free text.** For each hourly bar, Jev sees the computed market state and classifies the next few hours as "up," "down," or "flat" (a small dead-zone), with a probability for each. This makes the output directly

measurable — nothing is judged on how confident-sounding the answer is, only on the actual numbers.

3. **Calibration is checked before anything else is trusted.** For every class, the system asks: "of all the times Jev said there was an 80% chance of X, did X actually happen about 80% of the time?" This is standard forecast-verification math (a Brier score), the same approach used to grade weather forecasters. A model that's often right by luck but badly calibrated — overconfident or underconfident — fails this check even if its raw accuracy looks fine. That's exactly the trap this system exists to catch.
4. **A persistence filter stands between "the AI said something" and "act on it."** A single high-confidence reading isn't enough — that could be noise. A signal only counts once Jev's probability clears a threshold (e.g. 85%) for several consecutive hourly reads in a row.
5. **Hysteresis and a cooldown prevent whipsaw.** Once "in a position," the system doesn't exit the instant confidence dips slightly — it waits for a real drop (a separate, lower warning threshold), then enforces a cooldown before considering a new entry. This mirrors how a careful trader avoids getting chopped up flipping in and out of the same noisy move.
6. **Long and short are symmetric, independent decisions.** *Long* = Jev is confident price is going up → buy now, sell later. *Short* = Jev is confident price is going down → sell now, buy back later at a lower price for the profit. Both run through the identical filter logic, just watching a different probability stream (P(up) vs. P(down)).
7. **Every simulated trade is priced in real money.** Given a starting capital amount, each entry→exit pair (or a still-open position, marked at the latest known price) becomes an actual percentage return and dollar P&L, compounding through a sequence of trades — not just "did it call the direction right," but "would this have grown or shrunk your account."
8. **Two ways to run it.**
    - **Backtest ("analyze")** — pick any historical date range; the system replays it bar-by-bar as if live, producing a calibration report and trade-by-trade P&L for that period. This is how the idea gets validated against history before it's trusted.
    - **Live ("poll")** — the identical filter logic, but with memory that persists between runs: each manual run only evaluates new market bars since the last one, and remembers whether it's currently "in a position." Deliberately kept manual, with no auto-scheduling, so there's never a surprise bill from an AI model being queried on autopilot.

What's been learned so far

Running this against real NIFTY market data, at the recommended (strict) settings, Jev's directional calls have **not yet cleared the bar**. Across a 3-month test and a separate 4-week test, the persistence filter found no qualifying trades at all — and more tellingly, Jev's stated confidence was **poorly calibrated**: its accuracy hovered near what you'd expect from random guessing among three outcomes, and every class scored worse than a naive "always guess the average" baseline. Deliberately loosening the filter to force a few example trades through surfaced a small, statistically meaningless handful — a mix of one loss and some gains — useful for confirming the mechanics work correctly, not for judging the model.

The honest current verdict: the system is correctly declining to trust Jev's signal yet. That's it working as designed — it exists specifically to prevent trading on an AI's confidence before that confidence has been shown to be trustworthy.

What this is *not*

- **Not a live trading bot** — it never places a real order.
- **Not options or derivatives** — this is a plain directional bet on the underlying asset's price, nothing more exotic.
- **Not automated** — nothing in live mode runs on a schedule; every check that spends an AI query is manually triggered, so cost and timing are always in your control.
- **Not a finished trading strategy** — it's a measurement instrument for deciding whether Jev (or any model) is worth trusting with real capital, and how a specific entry/exit discipline would have performed if you had trusted it.

Architecture, in one paragraph

A small Python service fetches real hourly market data (free, via Yahoo Finance, or a broker export), computes the market state, and calls Jev's API for a directional prediction. A statistics module grades those predictions against what actually happened. An entry/exit state machine applies the persistence/hysteresis/cooldown rules identically whether running a historical backtest or a live poll — the same code path, so backtested and live behavior can never quietly drift apart. Results are available three ways: a command-line tool, a local web dashboard (charts, trade tables, a live-status panel), and a JSON API underneath both — built so it could one day become a shared, multi-user service, though it isn't exposed to anyone else today.